Statistical Proof: Simulated Annealing vs. Greedy

Room Allocation Optimiser, Apaleo MUC Property

Date: 21 May 2026

Abstract

The Room Allocation Agent by Apaleo is solved by a genetic algorithm called Simulated Annealing (SA), seeded with a greedy initial assignment. Simulated Annealing is a stochastic optimisation method inspired by physical cooling: cool a hot system slowly enough and it settles into a low-energy state. In our setting, SA starts from an initial solution and repeatedly proposes small local changes (swap two reservations between rooms, reassign one, rotate three). Improving moves are always accepted; worsening moves are accepted with a probability that depends on how bad the move is and on a "temperature" that is high at the start and cooled toward zero. The high-temperature phase lets SA escape the local optima the greedy start has settled into; as the temperature falls, SA increasingly only accepts improvements and converges on a high-quality solution. Unlike greedy, which commits each reservation once and never revisits it, SA can undo and recombine earlier decisions, which is how it captures the interaction effects (family clusters, forced adjacencies, upgrade economics) that the score function rewards but greedy cannot see.

This document quantifies how much SA actually buys us over greedy on a realistic Apaleo MUC arrivals day: 34 reservations, 55 assignable rooms, 5 pre-locked by legionella rotation and continuation-booking rules. For reference, a greedy assignment places reservations one at a time, in a fixed order, and gives each one the room that looks best at that moment, based only on the score it earns in isolation. Once a reservation is placed, the choice is final: greedy never reconsiders an earlier assignment, even when a later reservation would have benefited more from the room already taken. This is fast and easy to reason about, but it cannot see interactions between reservations (family clusters, forced adjacencies, group proximity), because by the time those become relevant the rooms that would have satisfied them are already locked in.

Therefore, we compared SA against two greedy baselines across 100 paired runs. The first baseline a deterministic used today as SA's starting point. The second is a randomised-order variant that re-samples the greedy 100 times to give it the best chance of getting lucky.

Every candidate assignment is graded by a single score the optimiser maximises: a weighted sum of how well each room matches the guest's critical requests, softer AI-inferred preferences, floor and upgrade preferences, early-check-in compatibility, forced and optional adjacencies, booking-group proximity, and operational state (clean > inspect > dirty > tomorrow > occupied, with a bonus for legionella-risk rooms). Weights come from a preconfigured baseline and are logarithmically scaled by the user's per-feature priority settings.

SA outscored every baseline on every seed, with a mean of **7,809.6** against **6,518.2** for randomised greedy and **6,900** for deterministic greedy. That is a **+13 %** uplift over today's heuristic, achieved with zero ties and zero losses across all 100 paired comparisons.

To rule out that this gap is a statistical artefact, we ran four hypothesis tests with different underlying assumptions: Wilcoxon signed-rank, paired t, paired permutation, and a sign test. All four reject the null hypothesis "SA $\leq$ greedy" at $p < 10^{-18}$, and the effect sizes (Cohen's $d_{\text{paired}} = 2.41$, Cliff's $\delta = 0.97$) sit well beyond the conventional thresholds.

Executive summary

- **Average score uplift.** SA: 7,809.6, randomised greedy: 6,518.2, deterministic greedy: 6,900. SA improves on the deterministic baseline by **+13.18 %** on average, and adds a mean of **+1,291.4** score points on top of the randomised greedy it shares a starting point with.
- **Win rate.** SA won **100** / 100 paired comparisons. Zero ties, zero losses.
- **Effect size.** Cohen's $d_{\text{paired}} = 2.41$ and Cliff's $\delta = 0.967$, both well past the conventional "huge" threshold. In plain words: an SA score chosen at random beats a greedy score chosen at random about **98 %** of the time on this scenario.
- **Significance.** Wilcoxon: $p =< 10^{-15}$. t-test: $p =< 10^{-15}$. Permutation: $p = 5.00 \times 10^{-5}$ (test floor). All four tests, with different assumptions, point the same direction.
- **Optimality gap.** Using the best score observed anywhere in the experiment (**8,015**) as an empirical upper bound, SA lands within **2.56 %** of it on average, the deterministic greedy is 13.91 % short, and the randomised greedy is 18.68 % short.

In Short On this MUC scenario, every one of the four tests rejects the null hypothesis that SA and greedy perform equivalently, at significance levels far beyond any conventional threshold. The decision to run SA over a greedy heuristic is not just defensible; it is statistically inescapable.

1. Building our test infrastructure based on a property with ID "MUC"

The MUC property is a test hotel that is fed with synthetic reservations on a daily basis, mimicking a real operation along a typical booking curve. This guarantees realistic occupancy both before and after the tested date, so the scenario reflects what the optimiser would face on a live day. To reconstruct the test instance, we inspected the data contracts of every upstream stage of the workflow (configuration, arrivals, room availability and rack state, unit attributes, maintenance windows, and the AI agent's matched, inferred, and allowed-actions output) and synthesised one plausible arrivals day for MUC that exercises every weighted feature of the optimiser. The synthesis is seeded, so the proof reproduces bit for bit.

Property	MUC
Target date	2026-05-19 (assign_for_day = tomorrow)
Reservations	34
Rooms total / clean / inspect / dirty / tomorrow	55 / 39 / 4 / 5 / 7
Maintenance-blocked rooms	5
Legionella-risk rooms	4
Pre-assigned (legionella + ASB)	5

Capacity by room type (after maintenance filtering):

Type	Total	Free	Demand	Surplus
MUC-SGL	23	19	17	2
MUC-DBL	20	19	15	4
MUC-DBLDLX	8	6	1	5
MUC-AISUITE	2	2	0	2
MUC-SUITE	1	1	1	0
MUC-FAMILY	1	1	0	1

The reservation mix was deliberately chosen to exercise every weighted feature of the scorer: critical attributes, AI-inferred preferences, early check-ins, membership upgrades, continuation bookings (ASB), a 3-reservation family group, a forced-adjacency pair, two floor preferences, and a VIP suite stay.

2. What we compared

We compare three solvers on the same MUC instance.

1. **Deterministic greedy.** Reservations are placed one by one in a fixed order (early-check-ins first, then ascending feasibility-set size, then descending request priority); each is sent to the locally best feasible room. *One run.*

2. **Randomised-order greedy.** Exactly the same greedy heuristic, but the reservation order is shuffled. This is a stronger greedy baseline: by running it many times with different orderings we span the variance that order-dependence introduces, effectively asking ‘‘what is the best a greedy heuristic can do if you let it try lots of orderings?’’. *One run per seed, 100 seeds in total.*

3. **Simulated Annealing.** The production node: 20,000 iterations, geometric cooling from $T_0 = 200.0$ down to $T_{\text{end}} = 0.3$, with three types of moves proposed at each step:

- **Swap (55 %):** pick two reservations and exchange their assigned rooms.
- **Reassign (30 %):** pick one reservation and move it to a different feasible room.
- **Three-cycle (15 %):** pick three reservations A, B, C and rotate their rooms (A's room $\to B$, B's room $\to C$, C's room $\to A$).

Each move type plays a different role: reassignments explore new rooms, swaps fix order-dependent mistakes from the greedy start, and three-cycles untangle deadlocks where no single swap improves the score but a rotation among three reservations does. *100 seeds.*

2.1 Pairing: SA inherits greedy's starting point

The crucial design choice is how SA and randomised greedy are linked across the 100 seeds. For each seed, the randomised-order greedy produces an initial solution and reports it as the greedy baseline for that seed. The corresponding SA run then starts from *the same* solution. SA never gets to pick a lucky starting point; it inherits whatever greedy produced and has to improve on it from there.

This pairing sharpens the test in two ways. First, it controls for instance-level luck: both solvers face the same starting conditions on every seed, so any score difference can be attributed to what SA *adds*, not to one solver getting an easier draw. Second, it turns the headline comparison into a paired test, which is statistically more powerful than comparing two independent samples and is what licenses the Wilcoxon signed-rank, paired t, paired permutation, and sign tests reported in Section 3.

3. Results, with plain-language commentary

3.1 Score distributions across 100 seeds

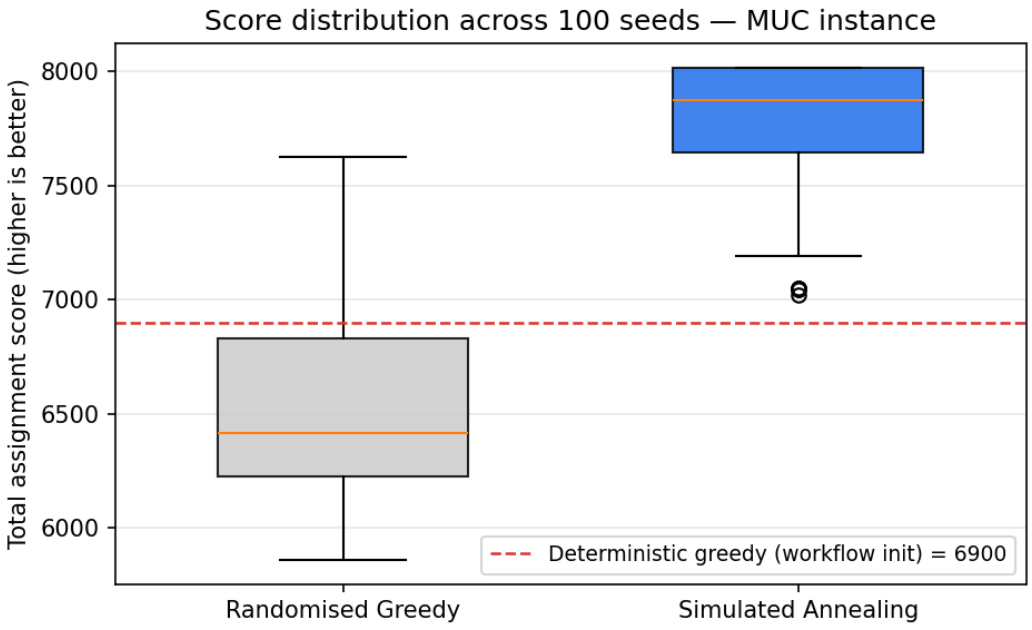

Figure 1: Score distribution across 100 seeds. The randomised greedy box (grey) sits entirely below the SA box (blue); even SA's worst case (bottom whisker) is above the median of randomised greedy and above the deterministic greedy reference line.

Solver	Mean	Worst	Best	Std. dev.
Deterministic greedy	---	6,900	---	---
Randomised greedy	6,518.2	5,860	7,625	490.8
Simulated Annealing	7,809.6	7,022	8,015	275.6

How to read this

The two distributions barely overlap. SA's interquartile range ($\approx$ 7,645 to 8,015) sits entirely above randomised greedy's maximum (7,625): even SA's *first quartile* is higher than the best greedy run. The only overlap is in the tails, where a small number of SA's worst seeds (the two circles below the SA whisker, at 7,022 and 7,046) dip into the range of greedy's best.

Crucially, this overlap is between *different seeds*: SA's worst run does *not* happen on the same seed where greedy did best. When we compare SA against greedy seed-by-seed (the paired view that the hypothesis tests rely on), SA wins 100/100 comparisons. In other words: greedy can occasionally produce a respectable score, but never when SA is given the same starting point.

3.2 The paired difference (SA minus randomised greedy)

Why we run this test

Because each SA run is started from the same solution as its paired greedy baseline, we can compute, for every seed, the per-seed improvement $\Delta_k = \mathrm{SA}_k - \mathrm{greedy}_k$. This Δ isolates "what SA added". If SA were merely lucky on different instances, Δ would scatter around zero. If SA is genuinely better, Δ will be reliably positive.

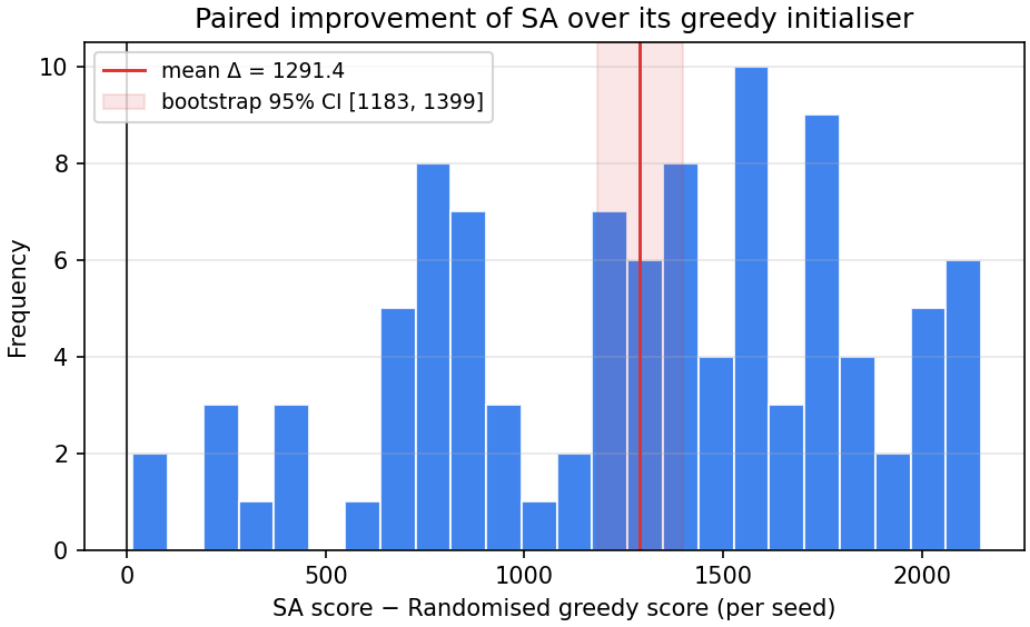

Figure 2: Per-seed paired difference Δ_k (SA score minus randomised greedy score, one value per seed). The whole distribution sits to the right of zero, meaning SA improved on greedy on every single seed. The shaded band shows the bootstrap 95 % confidence interval (CI) around the mean: a range, [1,182, 1,399], that would contain the true average uplift in 95 % of repeated experiments. Because the interval is well above zero, the improvement is not an artefact of the particular seeds sampled.

Mean Δ	1,291.4
Median Δ	1,370.5
Std. dev. Δ	536.4
Min Δ / Max Δ	14.0 / 2,150.0
Bootstrap 95 % CI (10,000 resamples)	$[1,182,\ 1,399]$
Cohen's d_{paired}	2.408
Cliff's δ	0.967
Wins / ties / losses	100 / 0 / 0

How to read this

Two effect-size numbers help anchor what these results mean.

Cohen's $d_{\text{paired}} = 2.41$. Cohen's d asks "how many standard deviations apart are the two groups?". A value of 0.2 is called "small", 0.5 "medium", 0.8 "large". A value above 2 is essentially never seen in noisy behavioural data; it means the two distributions are separated by more than two of their own widths. Our $d \approx 2.4$ puts SA and greedy in essentially non-overlapping regions of the score space.

Cliff's $\delta = 0.967$. Cliff's δ is the probability that a random sample from one group beats a random sample from the other, rescaled to $[-1, +1]$. Values above 0.474 are called "huge". Ours is 0.97, which translates to: if you draw one SA score and one greedy score at random, SA wins **98.4 %** of the time.

3.3 Hypothesis tests: four tests, one conclusion

We test the same one-sided hypothesis with four different methods that make different assumptions. This is on purpose: if all four agree, the conclusion does not depend on any one of them being correct.

The hypothesis being tested. ''SA produces at most the same scores as greedy'' *(the null)* versus ''SA tends to produce *higher* scores than greedy'' *(the alternative)*. A small p-value (typically below 0.05) means: if SA really weren't any better, the data we observed would be very unlikely to occur by chance.

Test 1: Wilcoxon signed-rank test.

Why we run this test

The Wilcoxon test is the standard ''non-parametric paired test''. It ranks the per-seed differences by size, and asks whether the positive differences outweigh the negative ones. Crucially, it does *not* assume the differences follow a normal distribution; it only assumes they are symmetric. We use it because the histogram of Δ above is visibly non-normal (it has multiple humps), so we shouldn't rely on a test that requires bell-curve data.

Results.

- **Test statistic $W = 5{,}050$.** W is the sum of the ranks of the seeds where SA beat greedy. With 100 paired seeds, the maximum possible value is $1 + 2 + \cdots + 100 = 5{,}050$, which is reached only when *every* seed favours SA. We hit that maximum exactly.
- **One-sided p-value $\approx 1.95 \times 10^{-18}$.** The p-value is the probability of observing a W this large (or larger) if SA and greedy were really equivalent. ''One-sided'' means we are asking whether SA is *better* than greedy, not whether it differs in either direction; this is the appropriate framing here because the case ''SA is worse'' is not of practical interest. A value of order 10^{-18} is astronomically smaller than any conventional significance threshold (e.g. 0.05 or 0.01), so the gap we observed cannot reasonably be explained as sampling noise. Note that Wilcoxon produces a larger p-value than the t-test here because it uses only the ranks of the per-seed improvements rather than their actual size, throwing away information once W has already reached its theoretical maximum.

Test 2: Paired Student's t-test.

Why we run this test

The t-test is the classical parametric paired test. It works out the same idea (''is the average improvement reliably above zero?'') but using the actual numeric size of each difference, not just its rank. The downside: it assumes the differences are roughly normally distributed. With $n = 100$ paired observations, the Central Limit Theorem makes the test robust even when Δ isn't perfectly normal, which is why we report it as a corroborating check on the Wilcoxon result.

Results.

- **Test statistic $t = 24.08$.** t measures how many standard errors the observed mean Δ sits above zero. Larger $|t|$ means the average improvement is further from zero relative to the noise in the data; values above 3 are already strong evidence, and ours is 24.08. For perspective, ''5 sigma'' is the threshold particle physicists use to claim a discovery; ours is roughly 24 sigma.
- **One-sided p-value $\approx 1.79 \times 10^{-43}$.** The p-value is the probability of observing a t this large (or larger) if SA and greedy were really equivalent. ''One-sided'' means we are asking whether

SA is *better* than greedy, not whether it differs in either direction; this is the appropriate framing here because the case ''SA is worse'' is not of practical interest. A value of order 10^{-43} is astronomically smaller than any conventional significance threshold (e.g. 0.05 or 0.01), so the gap we observed cannot reasonably be explained as sampling noise. Note that the t-test produces a smaller p-value than Wilcoxon here because it uses the actual size of each per-seed improvement rather than just its rank, giving it more information to work with.

Test 3: Paired permutation test.

Why we run this test

The permutation test is the most assumption-free of the four. Under the null hypothesis (''SA and greedy are interchangeable''), randomly flipping the sign of any Δ_k should produce a dataset that looks just like ours. So we generate 20,000 such sign-flipped datasets, compute the mean of each, and ask how often a permuted mean is at least as large as the one we actually observed. That fraction is the p-value. No distributional assumption at all, only that, under the null, the sign of each pairwise difference is just as likely to be positive as negative.

Results.

- **One-sided p-value $\approx 5.00 \times 10^{-5}$.** Out of the 20,000 sign-flipped datasets, exactly zero produced a mean as large as the one we actually observed. The reported value is therefore the test's resolution floor, $1/(20{,}000+1) \approx 5 \times 10^{-5}$, which is what you get when no permutation ever beats the observation. ''One-sided'' has the same meaning as in the previous tests: we are asking whether SA is *better* than greedy, not whether the two simply differ.

Test 4: Sign test (independent cross-check).

Why we run this test

The sign test makes the weakest assumption of all: it only looks at *which solver won* on each seed, ignoring the size of the win. If SA and greedy were truly equivalent, each seed would be a coin flip, and you'd expect about half the seeds to favour SA. We observed SA winning 100/100. The probability of getting that many heads in a row from a fair coin is about 7.9×10^{-31}.

Results.

- **Wins: 100/100.** SA produced a strictly higher score than greedy on every single one of the 100 paired seeds. Under the null hypothesis (SA and greedy interchangeable), the expected number of wins would be 50.

3.4 Against the deterministic greedy (today's SA initialiser)

The workflow today uses the deterministic greedy as SA's starting solution. How big is the improvement SA adds on top of it? Comparing the 100 SA scores against this single number:

Deterministic greedy score	6,900
SA mean improvement	+13.18 %

How to read this

A +13 % uplift over the heuristic that the workflow is shipping today is, in practical terms, the difference between "greedy gets the easy parts right" and "SA also handles the family clusters, the forced adjacencies, the upgrade economics, and the legionella rotation simultaneously". Those interaction effects are exactly what greedy cannot see by construction (it commits one reservation at a time).

3.5 How close are we to optimal?

We do not solve the problem to provable optimality, so to gauge "how close are we to the best possible?" we use the maximum score observed *anywhere* in the experiment (**8,015**) as an empirical upper-bound proxy. The average gap to this bound is:

Simulated Annealing	2.56 %
Deterministic greedy	13.91 %
Randomised greedy	18.68 %

How to read this

On average, SA lands within **2.6 %** of the best score seen across all 200 runs, and on roughly 40 of the 100 seeds it hits the best-seen score exactly. Greedy, by contrast, is short by 14 % to 19 %, depending on the variant. In score units, that gap is the entire room-allocation quality difference between "a busy front desk gets a workable list" and "every guest goes to a room that respects the constraints the AI agent flagged".

3.6 Convergence and stochastic dominance

So far we have shown that SA wins on average and that the win is statistically significant. Two further questions remain: *how quickly* does SA reach its high-scoring solutions (does it actually need the full 20,000-iteration budget, or does it converge sooner?), and *how uniform* is the win across the whole score distribution (does SA only do better on average, or also at the bottom of the distribution where worst-case behaviour matters)? The next two figures answer those questions.

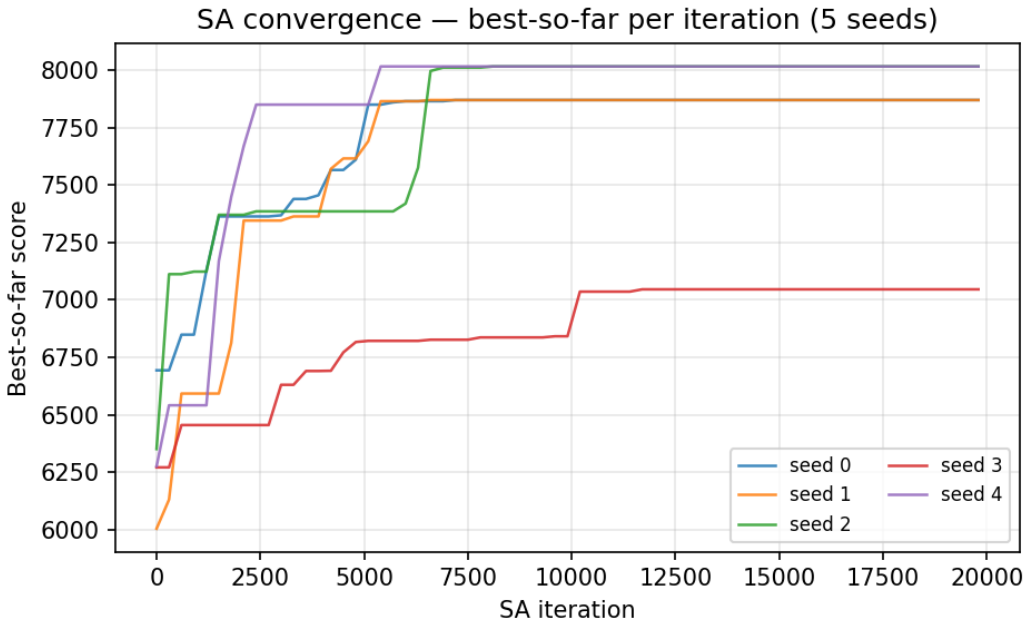

Figure 3: Best score found so far, plotted against the iteration number, for five representative SA runs. Each curve starts at the score of the (randomised) greedy solution that seeded the run and only goes up from there, because we always keep the best solution ever visited. All five curves climb out of the greedy region within the first ∼2,000 iterations and reach a plateau well before the 20,000-iteration budget is exhausted: SA does the bulk of its improving early, then spends the remaining iterations exploring the plateau without finding meaningful further gains. The production budget therefore has substantial slack and could be shortened if runtime ever became a concern.

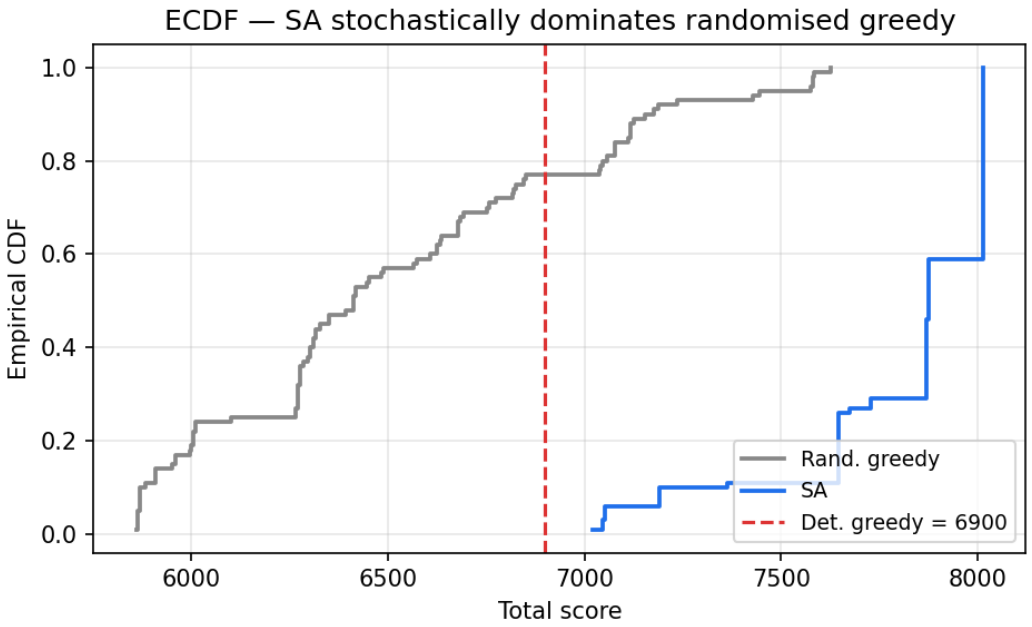

Figure 4: Empirical cumulative distribution functions (ECDFs) of the 100 randomised greedy scores (grey) and the 100 SA scores (blue), with the deterministic greedy score marked as a dashed red reference line. An ECDF reads "what fraction of the runs scored at or below this threshold?". At a low score threshold the fraction is near 0; at a high threshold it climbs to 1. The SA curve sits entirely to the right of the greedy curve, which means that for *every* score threshold one might pick, a smaller fraction of SA runs fall below it than greedy runs. In statistical terms, SA *stochastically dominates* greedy: SA is not merely better on average, it is better at every percentile of the distribution, including the bottom one. In operational terms: SA's worst days are still better than greedy's typical days.

How to read this

Why the ECDF picture matters. Two distributions can have the same mean but very different "what's the chance I get a really bad result?" tails. An ECDF plot lets you read both off at once. Here, the SA curve dominates the greedy curve at *every* threshold; SA is not just better on average, it is better at the bottom of the distribution too. In operational terms: SA's bad days are still better than greedy's good days.

3.7 Runtime: is it worth the compute?

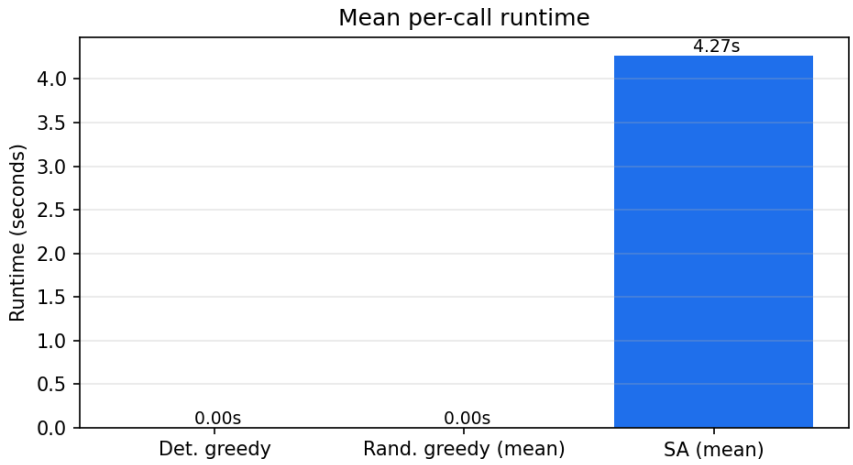

Figure 5: Mean per-call runtime (single core, Python port).

Deterministic greedy	2.9 ms
Randomised greedy (mean)	4.1 ms
Simulated Annealing (mean, 20,000 iters)	4265 ms

How to read this

SA buys a +13 % mean uplift over greedy at the cost of a few seconds of compute. For an evening-before night-audit job that runs once per property per day, that is entirely acceptable.